

How we can avoid unnecessary repetitions for recursive calls?

We can avoid this unnecessary repetitions by writing down the results of recursive calls and looking them up again if we need them later. This process is called memoization

Question No: 19 (Marks: 2)

Draw the cost table for chain matrix multiplication problem with initial state.

$(A_1)(A_2A_3A_4 \dots A_n)$

or $(A_1A_2)(A_3A_4 \dots A_n)$

or $(A_1A_2A_3)(A_4 \dots A_n)$

.....

or $(A_1A_2A_3A_4 \dots A_{n-1})(A_n)$

What are Catalan numbers? Give the formula.

PAGE NO. 85 ON PDF

Heap sort Algorithm

```
HEAPSORT( array A, int n)
1  BUILD-HEAP(A, n)
2  m ← n
3  while (m ≥ 2)
4  do SWAP(A[1], A[m])
5     m ← m - 1
6     HEAPIFY(A, 1, m)
```

Page #41

BFS pseudo code in graph

Start with S and visit its adjacent nodes. Label them with distance 1. Now consider the neighbours of neighbours of S. These would be at distance 2. Now consider the neighbours of neighbours of neighbours of S. These would be at distance 3. Repeat this until no more unvisited neighbours left to visit.

heapify

There is one principal operation for maintaining the heap property. It is called Heapify. The idea is that we are given an element of the heap which we suspect may not be in valid heap order, but we assume that all of other the elements in the subtree rooted at this element are in heap order. In particular this root element may be too small. To fix this we “sift” it down the tree by swapping it with one of its children. Which child? We should take the larger of the two children to satisfy the heap ordering property. This continues recursively until the element is either larger than both its children or until it falls all the way to the leaf level.

Polynomial time algorithm

A *polynomial time algorithm* is any algorithm that runs in $O(n^k)$ time. A problem is solvable in polynomial time if there is a polynomial time algorithm for it.

What is a run time analysis and its two criteria

The main purpose of our mathematical analysis will be measuring the execution time. The running time of an implementation of the algorithm would depend upon the speed of the computer, programming language, optimization by the compiler etc. Two criteria for measuring running time are *worst-case time* and *average-case time*.

Worst-case time is the maximum running time over all (legal) inputs of size n . Let I denote an input instance, let $|I|$ denote its length, and let $T(I)$ denote the running time of the algorithm on input I . Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

Average-case time is the average running time over all inputs of size n . Let $p(I)$ denote the probability of seeing this input. The average-case time is the weighted sum of running times with weights being the probabilities:

$$T_{\text{avg}}(n) = \sum_{|I|=n} p(I)T(I)$$

We will almost always work with worst-case time. Average-case time is more difficult to compute; it is difficult to specify probability distribution on inputs. Worst-case time will specify an upper limit on the running time.

Page #13

Dijkstra algorithm correctness criteria two conditions

We will prove the correctness of Dijkstra's algorithm by Induction. We will use the definition that $\delta(s, v)$ denotes the minimal distance from s to v .

For the base case

1. $S = \{s\}$
2. $d(s) = 0$, which is $\delta(s, s)$

Question No: 50 (Marks: 5)

Let the adjacency list representation of an undirected graph is given below. Explain what general property of the list indicates that the graph has an isolated vertex.

$a \rightarrow b \rightarrow c \rightarrow e$

$b \rightarrow a \rightarrow d$

$c \rightarrow a \rightarrow d \rightarrow e \rightarrow f$

$d \rightarrow b \rightarrow c \rightarrow f$

$e \rightarrow a \rightarrow c \rightarrow f$

$f \rightarrow c \rightarrow d \rightarrow e$

g

If any of the vertex do not have any adjacent vertices, It is shown in the list a single entity.

Question No: 48 (Marks: 3)

Recursive explanation of dynamic programming.

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller sub problems.

- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

Question No: 28 (Marks: 2)

Differentiate between back edge and forward edge.

Back edge: (u, v) where v is an ancestor of u in the tree.

Forward edge: (u, v) where v is a proper descendent of u in the tree.

Question No: 29 (Marks: 2)

How the generic greedy algorithm operates in minimum spanning tree?

A generic greedy algorithm operates by repeatedly adding any *safe* edge to the current spanning tree.

Question No: 30 (Marks: 2)

What are two cases for computing assuming we already have the previous matrix $d^{(k-1)}$ using Floyd-Warshall algorithm?

There are two basic cases:

1. Don't go through vertex k at all.

2. Do go through vertex k .

Question No: 31 (Marks: 3)

Describe Minimum Spanning Trees Problem with examples.

A common problem in communications networks and circuit design is that of connecting together a set of nodes by a network of total minimum length. The length is the sum of lengths of connecting wires.

Question No: 32 (Marks: 3)

What is decision problem, also explain with example?

A problem is called a *decision problem* if its output is a simple “yes” or “no” (or you may think of this as true/false, 0/1, accept/reject.)

Question No: 33 (Marks: 3)

Prove that the generic TRAVERSE (S) marks every vertex in any connected graph exactly once and the set of edges $(v, \text{parent}(v))$ with $\text{parent}(v) \neq \text{null}$ form a spanning tree of the graph.

First, it should be obvious that no vertex is marked more than once. Clearly, the algorithm marks S. Let $v \neq S$ be a vertex and let $S = v_0, v_1, \dots, v_k = v$ be a path from S to v with the minimum number of edges.

Page #125 lemma

Question No: 35 (Marks: 5)

Prove the following lemma,

Lemma: Given a digraph $G = (V, E)$, consider any DFS forest of G and consider any edge $(u, v) \in E$. If this edge is a tree, forward or cross edge, then $f[u] > f[v]$. If this edge is a back edge, then $f[u] \leq f[v]$

Proof: For the non-tree forward and back edges the proof follows directly from the parenthesis lemma. For example, for a forward edge (u, v) , v is a descendent of u and so v's start-finish interval is contained within u's implying that v has an earlier finish time. For a cross edge (u, v) we know that the two time intervals are disjoint. When we were processing u, v was not white (otherwise (u, v) would be a tree edge), implying that v was started before u. Because the intervals are disjoint, v must have also finished before u.

1) RAM (Random Access memory) and its Applications?

A RAM is an idealized machine with an infinitely large random-access memory. Instructions are executed one-by-one (there is no parallelism). Each instruction involves performing some basic operation on two values in the machine's memory. Basic operations include things like assigning a value to a variable, computing any basic arithmetic operation (+, -, $\times$, integer division) on integer values of any size, performing any comparison (e.g. $x \leq 5$) or boolean operations, accessing an element of an array (e.g. $A[10]$). We assume that each basic operation takes the same constant time to execute.

2) Describe Dijkstra's algorithm working?

Dijkstra's algorithm works on a weighted directed graph $G = (V, E)$ in which all edge weights are non-negative, i.e., $w(u, v) \geq 0$ for each edge $(u, v) \in E$.

3) Prim algorithm graph?

Prim's algorithm builds the MST by adding leaves one at a time to the current tree. We start with a root vertex r ; it can be any vertex. At any time, the subset of edges A forms a single tree. We look to add a single vertex as a leaf to the tree. Graph on page #150.

4) Give a detailed example for 2-d maxima problem

Suppose you want to buy a car. You want to pick the fastest car. But fast cars are expensive; you want the cheapest. You cannot decide which is more important: speed or price. Definitely do not want a car if there is another that is both faster and cheaper. We say that the fast, cheap car *dominates* the slow, expensive car relative to your selection criteria. So, given a collection of cars, we want to list those cars that are not dominated by any other. Here is how we might model this as a formal problem.

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to be dominated by point q if $p.x \leq q.x$ and $p.y \leq q.y$.
- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be maximal if it is not dominated by any other point in P .

The car selection problem can be modelled this way: For each car we associate (x, y) pair where x is the speed of the car and y is the negation of the price. High y value means a cheap car and low y means expensive car. Think of y as the money left in your pocket after you have paid for the car. Maximal points correspond to the fastest and cheapest cars.

The 2-dimensional Maxima is thus defined as

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space, output the set of maximal points of P , i.e., those points p_i such that p_i is not dominated by any other point of P .

8) Pseudo code algorithm for DFS Timestamp?

DFS(G)

1 for (each $u \in V$)

2 do color[u] = white

3 pred[u] = nil

4 time = 0

5 for each $u \in V$

6 do if (color[u] = white)

7 then DFSVISIT(u)

(2) what is common problem in communication networks and circuit designing? 2

A common problem in communications networks and circuit design is that of connecting together a set of nodes by a network of total minimum length. The length is the sum of lengths of connecting wires.

(3) write pseudo code of relaxing a vertex 5

RELAX((u, v))

1 if ($d[u] + w(u, v) < d[v]$)

2 then $d[v] = d[u] + w(u, v)$

3 $pred[v] = u$

Page #155

(4) write pseudo code of dijkstra algorithm? 5

Page #156

(5) define NP completeness 5

The set of NP-complete problems is all problems in the complexity class NP for which it is known that if any one is solvable in polynomial time, then they all are. Conversely, if any one is not solvable in polynomial time, then none are.

Definition: A decision problem L is NP-Hard if

$L_0 \leq_p L$ for all $L_0 \in \text{NP}$.

Definition: L is NP-complete if

1. $L \in \text{NP}$ and

2. $L_0 \leq_p L$ for some known NP-complete problem L_0 .

Page #176

(6) define Floyd Warshall algorithm in these two cases 5

do not go through a vertex k at all

do go through a vertex k

Don't go through k at all

Then the shortest path from i to j uses only intermediate vertices $\{1, 2, \dots, k-1\}$. Hence the length of the shortest is $d_{(k-1)}$

||

Do go through k

First observe that a shortest path does not go through the same vertex twice, so we can assume that we pass through k exactly once. That is, we go from i to k and then from k to j . In order for the overall path to be as short as possible, we should take the shortest path from i to k and the shortest path from k to j . Since each of these paths uses intermediate vertices $\{1, 2, \dots, k-1\}$, the length of the path is

$d_{(k-1)}$

$i_k + d_{(k-1)}$

k_j .

Page #162

(7) Define DAG 3

A directed graph that is acyclic is called a *directed acyclic graph (DAG)*.

4 Questions of 5 Marks

1) Write steps of sieve techniques

We then *partition* A into three parts.

1. $A[q]$ contains the pivot element x .

2. $A[1..q - 1]$ will contain all the elements that are less than x and

3. $A[q + 1..n]$ will contain all elements that are greater than x .

Page #36

Prove the Lemma:

Consider a digraph $G = (V, E)$ and any DFS forest for G . G has a cycle if and only if the DFS forest has a back edge

Proof: If there is a back edge (u, v) then v is an ancestor of u and by following tree edge from v to u , we get a cycle.

Where the clique cover problem is used?

In clustering applications.

2) Analyze the brute force maxima algorithm

Assume that the input size is n , and for the running time we will count the number of times that any element of P is accessed. Clearly we go through the outer loop n times, and for each time through this loop, we go through the inner loop n times as well. The condition in the if-statement makes four accesses to P . The output statement makes two accesses for each point that is output. In the worst case every point is maximal so these two accesses are made for each time through the outer loop.

3) How Kruskal's algorithm works ?

Kruskal's algorithm works by adding edges in increasing order of weight (lightest edge first). If the next edge does not induce a cycle among the current set of edges, then it is added to A . If it does, we skip it and consider the next in order. As the algorithm runs, the edges in A induce a *forest* on the vertices. The trees of this forest are eventually merged until a single tree forms containing all vertices.

4) Which points should be supposed to prove the correctness of the Dijkstra's Algorithm

We will prove the correctness of Dijkstra's algorithm by Induction. We will use the definition that $_d(s, v)$

denotes the minimal distance from s to v .

For the base case

1. $S = \{s\}$

2. $d(s) = 0$, which is $_ (s, s)$

Page #158

What are two steps generally involved while developing a dynamic programming algorithm?

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller subproblems.

- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

What is the running time of the Dijkstra's algorithm?

$_ (E \log V)$

The following adjacency matrix represents a graph that consists of four vertices labeled 0, 1, 2 and 3. The entries in the matrix indicate edge weights.

	0	1	2	3
0	0	1	0	3
1	2	0	4	0
2	0	1	0	1
3	2	0	0	0

Answer the following question:

Can an adjacency matrix for a directed graph ever not be square in shape? Why or why not?

NO. Because number of rows and number of columns are always same.

What is the application of edit distance technique?

Edit Distance: Applications

There are numerous applications of the Edit Distance algorithm. Here are some examples:

Spelling Correction

If a text contains a word that is not in the dictionary, a 'close' word, i.e. one with a small edit distance, may be suggested as a correction. Most word processing applications, such as Microsoft Word, have spelling checking and correction facility. When Word, for example, finds an incorrectly spelled word, it makes suggestions of possible replacements.

Plagiarism Detection

If someone copies, say, a C program and makes a few changes here and there, for example, change variable names, add a comment of two, the edit distance between the source and copy may be small. The edit distance provides an indication of similarity that might be too close in some situations.

Computational Molecular Biology

Speech Recognition

Algorithms similar to those for the edit-distance problem are used in some speech recognition systems. Find a close match between a new utterance and one in a library of classified utterances.

7. What are the minimum and maximum numbers of elements in a heap of height h ?

$$\text{min} = 2^h$$

$$\text{max} = 2^{(h+1)} - 1$$

8. Where clique cover problem arises?

The clique cover problem arises in applications of clustering. We put an edge between two nodes if they are similar enough to be clustered in the same group. We want to know whether it is possible to cluster all the vertices into k groups.

QNo.1 What is heap and what is heap order? (Mark2)

Answer:-

The heap is the section of computer memory where all the variables created or initialized at runtime are stored. The heap order property: in a

(min) heap, for every node X , the key in the parent is smaller than or equal to the key in X .

Ref: [Handouts Page no. 40](#)

QNo.2 Quick sort such that sort the array in to non-increasing order? (Mark2)

Answer:-

Quick sorting, an array $A[1..n]$ of n numbers We are to reorder these elements into increasing (or decreasing) order. More generally, A is an array of objects and we sort them based on one of the attributes - the key value. The key value need not be a number. It can be any object from a totally ordered domain. Totally ordered domain means that for any two elements of the domain, x and y , either $x < y$, $x = y$ or $x > y$.

Ref: [Handouts Page no. 40](#)

QNo.3 Draw the cost table for chain multiplication problem with initial states(Mark3)

Answer:-

$(A1)(A2A3A4 \dots An)$

or $(A1A2)(A3A4 \dots An)$

or $(A1A2A3)(A4 \dots An)$

.....

or $(A1A2A3A4 \dots An-1)(An)$

Ref: [Handouts Page no. 90](#)

QNo.4 we can avoid unnecessary repetitions for recursive calls? (Mark3)

Answer:-

We can avoid these unnecessary repetitions by writing down the results of recursive calls and looking them up again if we need them later. This process is called memorization

Ref: [Handouts Page no. 49](#)

Worst case for edit distance algorithm? What is the simple change that can change the worst case time ? 5 marks

Answer:-

Analysis of DP edit distance

There are $\Theta(n^2)$ entries in the matrix. Each entry $E(i,j)$ takes $\Theta(1)$ time to compute. The total running is $\Theta(n^2)$ Recursion clearly leads to the same repetitive call pattern that we saw in Fibonacci sequence. To avoid this, we will use the DP approach. We will build the solution bottom-up. We will use the base case $E(0,j)$ to fill first row and the base case $E(i,0)$ to fill first column. We will fill the remaining E matrix row by row.

Ref: [Handouts Page no. 14](#)

Describe an efficient algorithm to find the median of a set of 10⁶ integers; it is known that there are fewer than 100 distinct integers in the set

Solution:-

Step1:Start

Step2:Find the 100 distinct numbers among 10⁶ integers.

Step3:Sort the 100 distinct numbers

Step4:Count the distinct numbers

Step5: if count is odd,middle number is the median

Step6:if count is even,add the middle two numbers then divide by 2,the result is the median

Step5:End
number.

[Ref: Handouts Page no. 34](#)

What is the formula for generating Catalan numbers?

Solution

Equation (22) is a recurrence relation.

$$C_{n+1} = C_n * [2(2n+1)] / (n+2)$$

we have the values of n in one column and the values of C_n in another, then to put this formula in Excel, on the (n+1)-th row just replace C_n and n with the appropriate cells from the previous row.

[Ref: Handouts Page no. 85](#)

What are Catalan numbers? Give the formula.

Catalan numbers form a sequence of natural numbers that occur in various counting problems, often involving recursively defined objects

Formula is $C(n) = \frac{2n}{n+1} C_{n-1}$

[Ref: Handouts Page no. 85](#)

Q-Write a pseudo code Fibonacci With memorization? -- (3)

Sol

MEMOFIB(n)

1 if ($n < 2$)

2 then return n

3 if (F[n] is undefined)

4 then F[n] = MEMOFIB(n - 1) + MEMOFIB(n - 2)

5 return F[n]

Ref: [Handouts Page no. 12, 74](#)

Q – Write Down the steps of Dynamic programming (5)

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- Formulate problem recursively. Write down a formula for the whole problem as a simple combination of answers to smaller sub problems.
- Build solution to recurrence from bottom up. Write an algorithm that starts with base cases and works its way up to the final solution.

Dynamic programming algorithms need to store the results of intermediate sub problems. This is often but not always done with some kind of table. We will now cover a number of examples of problems in which the solution is based on dynamic programming strategy.

Ref: [Handouts Page no. 74 – 77](#)

How we build heap

We build a max heap out of the given array of numbers A[1..n]. We repeatedly extract the maximum item from the heap. Once the max item is removed, we are left with a hole at the root.

Ref: [Handouts Page no. 41](#)

Write Pseudo code for KNAPSACK algorithm? 5 marks

Solution:

KNAPSACK (n, W)

1 for w=0,W

2 do $V[0,w] \leftarrow 0$

3 for i=0,n

4 do $V[i,0] \leftarrow 0$

5 for w=0,W

6 do if($w_i \leq w$ & $v_i + V[i-1, w - w_i] > V[i-1, w]$)

7 then $V[i, w] \leftarrow v_i + V[i-1, w]$

8 else $V[i, w] \leftarrow V[i-1, w]$

The time complexity is clearly $O(n, W)$, it must be cautioned that as n and W get large, both time and space complexity become significant.

Ref: [Handouts Page no. 96](#)

Spelling correction in edit distance? 3 marks

A better way to display this editing process is to place the words above the other:

S D I M D M

M A - T H S

A - R T - S

THE FIRST WORD HAS A GAP FOR EVERY INSERTION (I) AND THE SECOND WORD HAS A GAP FOR EVERY DELETION (D). MATHES (M) DO NOT COUNT. THE EDIT TRANSCRIPT IS DEFINED AS A STRING OVER THE ALPHABET $\{M, S, I, d\}$ THAT DESCRIBES A TRANSFORMATION OF ONE STRING INTO OTHER. FOR EXAMPLE

S D I M D M

$1+1+1+0+1+0=4$

Ref: [Handouts Page no. 77](#)

Differentiate b/w Bubble sort, insertion sort and selection sort? 3 marks

SOLUTION:

Bubble sort: scan the array. Whenever two consecutive items are found that are out of order, swap them. Repeat until all consecutive items are in order.

Insertion sort: assume that $A[1..i-1]$ have already been sorted. Insert $A[i]$ into its proper position in this sub array. Create this position by shifting all larger elements to the right.

Selection sort:

Assume that $A[1..i-1]$ contain the $i-1$ smallest elements in sorted order. Find the smallest in $A[i..n]$ swap it with $A[i]$.

Ref: [Handouts Page no. 54](#)

Write down the steps of dynamic programming strategy. (2 marks)

Solution:

Developing a dynamic programming algorithm generally involves two separate steps:

1_ formulate problem recursively.

Write down a formula for the whole problem as a simple combination of answers to smaller sub problems.

2_ Build solution to recurrence from bottom up:

Ref: [Handouts Page no. 75](#)

Solve the recursion problem. (5marks.)

Solution:

Recursion clearly leads to the same repetitive call pattern that we saw in Fibonacci sequence. To avoid this, we will use the DP approach. We will build the solution bottom-up. We will use the base case $E(0,j)$ to fill first row and the base case $E(i,0)$ to fill first column. We will fill the remaining E matrix row by row.

If we trace through the recursive calls to MemoFib, we find that array $F[]$ gets filled from bottom up...i.e., first $F[2]$, then $F[3]$, and so on, upto $F[n]$. we can replace recursion with a simple for-loop that just fills up the array $F[]$ in that order.

- We are given an array of n elements of $x_1, x_2, x_3, \dots, x_n$, suggest best sorting algorithm of order $O(n)$. (5 marks).

Solution:

The main shortcoming of counting sort is that it is useful for small integers, i.e., $1 \dots k$ where k is small. If this were a million more, the size of the rank array would also be a million. Radix sort provides a nice work around this limitation by sorting numbers one digit at a time.



☺☹☹ **MUHAMMAD FAISAL** ☺☹☹

MIT 4th Semester

Al-Barq Campus (VGJW01) Gujranwala

faisalgrw123@gmail.com

Reference Short Questions for MID TERM EXAMS

CS502– Design and Analysis of Algorithms

Divide and Conquer Strategy (Page#27)

The ancient Roman politicians understood an important principle of good algorithm design (although they were probably not thinking about algorithms at the time). You divide your enemies (by getting them to distrust each other) and then conquer them piece by piece. This is called divide-and-conquer. In algorithm design, the idea is to take a problem on a large input, break the input into smaller pieces, solve the problem on each of the small pieces, and then combine the piecewise solutions into a global solution. But once you have broken the problem into pieces, how do you solve these pieces? The answer is to apply divide-and-conquer to them, thus further breaking them down. The process ends when you are left with such tiny pieces remaining (e.g. one or two items) that it is trivial to solve them. Summarizing, the main elements to a divide-and-conquer solution are

Divide: the problem into a small number of pieces

Conquer: solve each piece by applying divide and conquer to it recursively

Combine: the pieces together into a global solution.

Selection Problem (Page#34)

Suppose we are given a set of n numbers. Define the rank of an element to be one plus the number of elements that are smaller. Thus, the rank of an element is its final position if the set is sorted. The minimum is of rank 1 and the maximum is of rank n .

Consider the set: {5, 7, 2, 10, 8, 15, 21, 37, 41}. The rank of each number is its position in the sorted order.

position	1	2	3	4	5	6	7	8	9
Number	2	5	7	8	10	15	21	37	41

Sieve Technique (Page#34)

The reason for introducing this algorithm is that it illustrates a very important special case of divide-and-conquer, which I call the sieve technique. We think of divide-and-conquer as breaking the problem into a small number of smaller subproblems, which are then solved recursively. The sieve technique is a special case, where the number of subproblems is just 1.

The sieve technique works in phases as follows. It applies to problems where we are interested in finding a single item from a larger set of n items.

Slow Sorting Algorithms (Page#39)

There are a number of well-known slow $O(n^2)$ sorting algorithms. These include the following:

Bubble sort: Scan the array. Whenever two consecutive items are found that are out of order, swap them. Repeat until all consecutive items are in order.

Insertion sort: Assume that $A[1..i-1]$ have already been sorted. Insert $A[i]$ into its proper position in this sub array. Create this position by shifting all larger elements to the right.

Selection sort: Assume that $A[1..i-1]$ contain the $i-1$ smallest elements in sorted order. Find the smallest element in $A[i..n]$ Swap it with $A[i]$.

Heaps (Page#40)

A heap is a left-complete binary tree that conforms to the heap order. The heap order property: in a (min) heap, for every node X , the key in the parent is smaller than or equal to the key in X . In other words, the parent node has key smaller than or equal to both of its children nodes. Similarly, in a max heap, the parent has a key larger than or equal both of its children. Thus the smallest key is in the root in a min heap; in the max heap, the largest is in the root.

Quicksort (Page#46)

Our next sorting algorithm is Quicksort. It is one of the fastest sorting algorithms known and is the method of choice in most sorting libraries. Quicksort is based on the divide and conquer strategy.

Here is the algorithm:

```
QUICKSORT( array A, int p, int r)
1  if (r > p)
2    then
3      i ← a random index from [p..r]
4      swap A[i] with A[p]
5      q ← PARTITION(A, p, r)
6      QUICKSORT(A, p, q - 1)
7      QUICKSORT(A, q + 1, r)
```

In-place, Stable Sorting (Page#46)

An in-place sorting algorithm is one that uses no additional array for storage. A sorting algorithm is stable if duplicate elements remain in the same relative position after sorting.

9 3 3' 5 6 5' 2 1 3''	unsorted
1 2 3 3' 3'' 5 5' 6 9	stable sort
1 2 3' 3 3'' 5' 5 6 9	unstable

Bubble sort, insertion sort and selection sort are in-place sorting algorithms. Bubble sort and insertion sort can be implemented as stable algorithms but selection sort cannot (without significant modifications). Mergesort is a stable algorithm but not an in-place algorithm. It requires extra array storage. Quicksort is not stable but is an in-place algorithm. Heapsort is an in-place algorithm but is not stable.

Counting Sort (Page#57)

We will consider three algorithms that are faster and work by not making comparisons. Counting sort assumes that the numbers to be sorted are in the range 1 to k where k is small. The basic idea is to determine the rank of each number in final sorted array.

Recall that the rank of an item is the number of elements that are less than or equal to it. Once we know the ranks, we simply copy numbers to their final position in an output array.

Fibonacci Sequence (Page#73)

Suppose we put a pair of rabbits in a place surrounded on all sides by a wall. How many pairs of rabbits can be produced from that pair in a year if it is supposed that every month each pair begets a new pair which from the second month on becomes productive? Resulting sequence is 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, . . . where each number is the sum of the two preceding numbers.

This problem was posed by Leonardo Pisano, better known by his nickname Fibonacci (son of Bonacci, born 1170, died 1250). This problem and many others were in posed in his book Liber abaci, published in 1202. The book was based on the arithmetic and algebra that Fibonacci had accumulated during his travels. The book, which went on to be widely copied and imitated, introduced the Hindu-Arabic place-valued decimal system and the use of Arabic numerals into Europe. The rabbits problem in the third section of Liber abaci led to the introduction of the Fibonacci numbers and the Fibonacci sequence for which Fibonacci is best remembered today.

This sequence, in which each number is the sum of the two preceding numbers, has proved extremely fruitful and appears in many different areas of mathematics and science. The Fibonacci Quarterly is a modern journal devoted to studying mathematics related to this sequence. The Fibonacci numbers F_n are defined as follows:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Figure shows four levels of recursion for the call $\text{fib}(8)$:

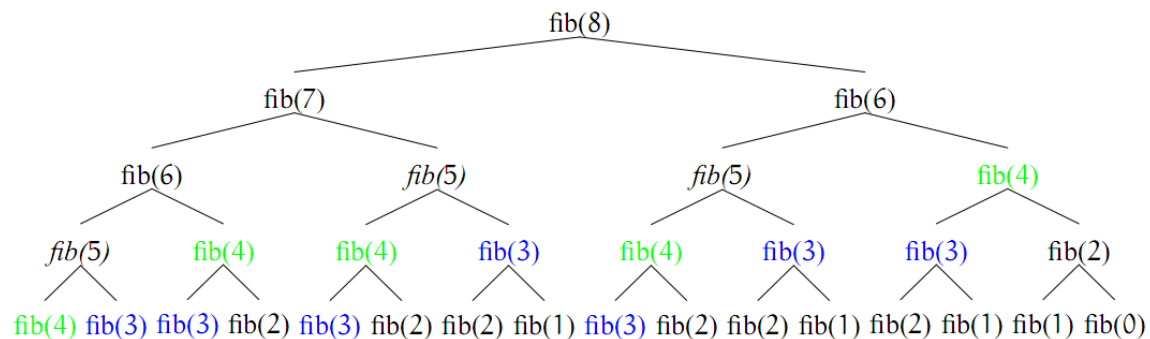


Figure 6.1: Recursive calls during computation of Fibonacci number

Dynamic Programming (Page#75)

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller subproblems.
- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

Dynamic programming algorithms need to store the results of intermediate subproblems. This is often but not always done with some kind of table. We will now cover a number of examples of problems in which the solution is based on dynamic programming strategy.

Edit Distance (Page#75)

The words “computer” and “commuter” are very similar, and a change of just one letter, p→m, will change the first word into the second. The word “sport” can be changed into “sort” by the deletion of the ‘p’, or equivalently, ‘sort’ can be changed into ‘sport’ by the insertion of ‘p’. The edit distance of two strings, s1 and s2, is defined as the minimum number of point mutations required to change s1 into s2, where a point mutation is one of:

- change a letter,
- insert a letter or
- delete a letter

For example, the edit distance between FOOD and MONEY is at most four:

$$\begin{array}{l} \underline{F}OOD \longrightarrow MO\underline{O}D \longrightarrow MON \underset{\wedge}{D} \\ \hspace{15em} \longrightarrow MONED \underline{D} \longrightarrow MONEY \end{array}$$

Edit Distance: Applications (Page#76)

There are numerous applications of the Edit Distance algorithm.

Here are some examples:

1. Spelling Correction
2. Plagiarism Detection
3. Computational Molecular Biology
4. Speech Recognition

Edit Distance Algorithm (Page#77)

A better way to display this editing process is to place the words above the other:

	<i>S</i>	<i>D</i>	<i>I</i>	<i>M</i>	<i>D</i>	<i>M</i>
<i>M</i>	A	_	T	H	S	
<i>A</i>		_	R	T	_	S

The first word has a gap for every insertion (I) and the second word has a gap for every deletion (D). Columns with two different characters correspond to substitutions (S). Matches (M) do not count. The Edit transcript is defined as a string over the alphabet M, S, I, D that describes a transformation of one string into another.

In general, it is not easy to determine the optimal edit distance. For example, the distance between ALGORITHM and ALTRUISTIC is at most 6.

A	L	G	O	R	_	I	_	T	H	M
A	L	_	T	R	U	I	S	T	I	C

CHAIN MATRIX MULTIPLY (Page#85)

There is considerable savings achieved even for this simple example. In general, however, in what order should we multiply a series of matrices $A_1 A_2 \dots A_n$. Matrix multiplication is an associative but not commutative operation. We are free to add parenthesis the above multiplication but the order of matrices can not be changed. The Chain Matrix Multiplication Problem is stated as follows:

Given a sequence $A_1, A_2, \dots, A_n$ and dimensions $p_0, p_1, \dots, p_n$ where A_i is of dimension $p_{i-1} \times p_i$, determine the order of multiplication that minimizes the number of operations.

We could write a procedure that tries all possible parenthesizations. Unfortunately, the number of ways of parenthesizing an expression is very large. If there are n items, there are $n - 1$ ways in which outer most pair of parentheses can be placed.

$$\begin{aligned}
 & (A_1)(A_2A_3A_4 \dots A_n) \\
 \text{or } & (A_1A_2)(A_3A_4 \dots A_n) \\
 \text{or } & (A_1A_2A_3)(A_4 \dots A_n) \\
 & \dots \quad \dots \\
 \text{or } & (A_1A_2A_3A_4 \dots A_{n-1})(A_n)
 \end{aligned}$$

We do not want to calculate m entries recursively. So how should we proceed? We will fill m along the diagonals. Here is how. Set all $m[i, i] = 0$ using the base condition. Compute cost for multiplication of a sequence of 2 matrices. These are $m[1, 2], m[2, 3], m[3, 4], \dots, m[n - 1, n]$. $m[1, 2]$, for example is

$$m[1, 2] = m[1, 1] + m[2, 2] + p_0 \cdot p_1 \cdot p_2$$

For example, for m for product of 5 matrices at this stage would be:

$m[1, 1]$	$\leftarrow m[1, 2]$ $\downarrow$			
	$m[2, 2]$	$\leftarrow m[2, 3]$ $\downarrow$		
		$m[3, 3]$	$\leftarrow m[3, 4]$ $\downarrow$	
			$m[4, 4]$	$\leftarrow m[4, 5]$ $\downarrow$
				$m[5, 5]$

Knapsack Problem (Page#91)

A thief goes into a jewelry store to steal jewelry items. He has a knapsack (a bag) that he would like to fill up. The bag has a limit on the total weight of the objects placed in it. If the total weight exceeds the limit, the bag would tear open. The value of the jewelry items varies from cheap to expensive. The thief's goal is to put items in the bag such that the value of the items is maximized and the weight of the items does not exceed the weight limit of the bag. Another limitation is that an item can either be put in the bag or not - fractional items are not allowed. The problem is: what jewelry should the thief choose that satisfy the constraints?

Formally, the problem can be stated as follows: Given a knapsack with maximum capacity W , and a set S consisting of n items. Each item i has some weight w_i and value v_i (all w_i , v_i and W are integer values). How to pack the knapsack to achieve maximum total value of packed items? For example, consider the following scenario:



Item i	Weight w_i	Value v_i
1	2	3
2	3	4
3	4	5
4	5	8
5	9	10

Figure 6.3: Knapsack can hold $W = 20$

The knapsack problem belongs to the domain of optimization problems. Mathematically, the problem is

$$\begin{aligned} &\text{maximize } \sum_{i \in T} v_i \\ &\text{subject to } \sum_{i \in T} w_i \leq W \end{aligned}$$

The **steps** in the **dynamic programming strategy**: (Page#92)

- 1) Simple Subproblems
 - 2) Principle of Optimality
 - 3) Bottom-up computation
- Construction of optimal solution

Knapsack algorithm (Page#96)

```
KNAPSACK( $n, W$ )
1  for  $w = 0, W$ 
2  do  $V[0, w] \leftarrow 0$ 
3  for  $i = 0, n$ 
4  do  $V[i, 0] \leftarrow 0$ 
5    for  $w = 0, W$ 
6    do if ( $w_i \leq w$  &  $v_i + V[i - 1, w - w_i] > V[i - 1, w]$ )
7        then  $V[i, w] \leftarrow v_i + V[i - 1, w - w_i]$ ;  $\text{keep}[i, w] \leftarrow 1$ 
8        else  $V[i, w] \leftarrow V[i - 1, w]$ ;  $\text{keep}[i, w] \leftarrow 0$ 
9  // output the selected items
10  $k \leftarrow W$ 
11 for  $i = n$  downto 1
12 do if ( $\text{keep}[i, k] = 1$ )
13     then output  $i$ 
14          $k \leftarrow k - w_i$ 
```

-----Wish U Best of L|U|C|K for EXAMS -----

MUHAMMAD FAISAL

Al-Barq Campus (VGJW01) Gujranwala

VIRTUAL UNIVERSITY OF PAKISTAN

Cs 502 Subjective Mid Term

Prepared by

SAIF

FOR PAST PAPERS VISIT:

<http://vupkpastpapers.blogspot.com>

REMEMBER ME IN YOUR PRAYERS

MID TERM EXAMINATION

Heap sort Algorithm

HEAPSORT(array A , int n)

```
1  BUILD-HEAP( $A, n$ )
2   $m \leftarrow n$ 
3  while ( $m \geq 2$ )
4  do SWAP( $A[1], A[m]$ )
5      $m \leftarrow m - 1$ 
6     HEAPIFY( $A, 1, m$ )
```

Page #41

BFS pseudo code in graph

Start with S and visit its adjacent nodes. Label them with distance 1. Now consider the neighbours of neighbours of S . These would be at distance 2. Now consider the neighbours of neighbours of neighbours of S . These would be at distance 3. Repeat this until no more unvisited neighbours left to visit.

heapify

There is one principal operation for maintaining the heap property. It is called Heapify. The idea is that we are given an element of the heap which we suspect may not be in valid heap order, but we assume that all of other the elements in the subtree rooted at this element are in heap order. In particular this root element may be too small. To fix this we “sift” it down the tree by swapping it with one of its children. Which child? We should take the larger of the two children to satisfy the heap ordering property. This continues recursively until the element is either larger than both its children or until its falls all the way to the leaf level.

Polynomial time algorithm

A *polynomial time algorithm* is any algorithm that runs in $O(n^k)$ time. A problem is solvable in polynomial time if there is a polynomial time algorithm for it.

What is a run time analysis and its two criteria

The main purpose of our mathematical analysis will be measuring the execution time. The running time of an implementation of the algorithm would depend upon the speed of the computer, programming language, optimization by the compiler etc. Two criteria for measuring running time are *worst-case time* and *average-case time*.

Worst-case time is the maximum running time over all (legal) inputs of size n . Let I denote an input instance, let $|I|$ denote its length, and let $T(I)$ denote the running time of the algorithm on input I . Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

Average-case time is the average running time over all inputs of size n . Let $p(I)$ denote the probability of seeing this input. The average-case time is the weighted sum of running times with weights being the probabilities:

$$T_{\text{avg}}(n) = \sum_{|I|=n} p(I)T(I)$$

We will almost always work with worst-case time. Average-case time is more difficult to compute; it is difficult to specify probability distribution on inputs. Worst-case time will specify an upper limit on the running time. **Page #13**

Dijkstra algorithm correctness criteria two conditions

We will prove the correctness of Dijkstra's algorithm by Induction. We will use the definition that $\delta(s, v)$ denotes the minimal distance from s to v .

For the base case

1. $S = \{s\}$
2. $d(s) = 0$, which is $\delta(s, s)$

Page #158

Floyd algorithm

A graph analysis algorithm for finding shortest paths in a weighted graph with positive or negative edge weights. A single execution of the algorithm will find the lengths (summed weights) of the shortest paths between *all* pairs of vertices.

write pseudo code of dijkstra algorithm? 5marks

```
DIJKSTRA((G, w, s))
1  for ( each u ∈ V)
2  do d[u] ← ∞
3  pq.insert (u, d[u])
4  d[s] ← 0; pred[s] ← nil; pq.decrease_key (s, d[s]);
5  while ( pq.not_empty ())
6  do u ← pq.extract_min ()
7  for ( each v ∈ adj[u])
8  do if (d[u] + w(u, v) < d[v])
9  then d[v] = d[u] + w(u, v)
10     pq.decrease_key (v, d[v])
11     pred[v] = u
```

Page #156

Compare bellman ford algorithm with dijkstra's algorithm. Also give the time complexity of bellman ford algorithm (3 marks)

Dijkstra's single-source shortest path algorithm works if all edges weights are non-negative and there are no negative cost cycles. Bellman-Ford *allows* negative weights edges and no negative cost cycles. The algorithm is slower than Dijkstra's, running in $O(VE)$ time. Like Dijkstra's algorithm, Bellman-Ford is based on performing repeated relaxations. Bellman-Ford applies relaxation to *every edge* of the graph and repeats this $V - 1$ times.

Page #159

psedo code of timestamp DFS

```
DFS(G)
1 for (each u ∈ V)
2 do color[u] ← white
3 pred[u] ← nil
4 time ← 0
5 for each u ∈ V
6 do if (color[u] = white)
7 then DFSVISIT(u)
```

Page #126

variants of shortest path solution

There are a few variants of the shortest path problem.

Single-source shortest-path problem: Find shortest paths from a given (single) *source* vertex $s \in V$ to every other vertex $v \in V$ in the graph G .

Single-destination shortest-paths problem: Find a shortest path to a given destination vertex t from each vertex v . We can reduce this problem to a single-source problem by reversing the direction of each edge in the graph.

Single-pair shortest-path problem: Find a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u , we solve this problem also. No algorithms for this problem are known to run asymptotically faster than the best single-source algorithms in the worst case.

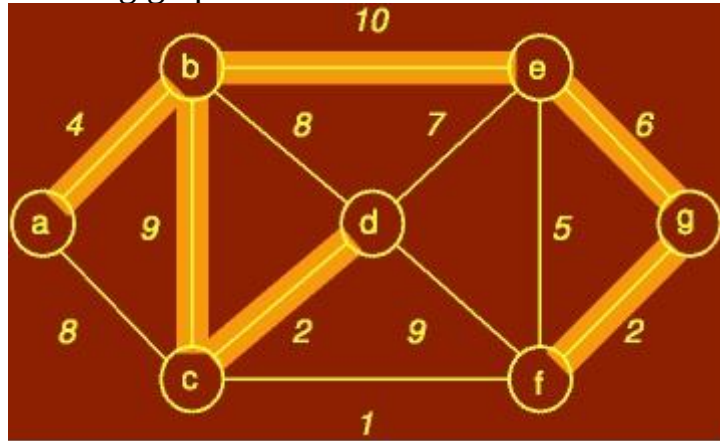
All-pairs shortest-paths problem: Find a shortest path from u to v for *every pair* of vertices u and v . Although this problem can be solved by running a single-source algorithm once from each vertex, it can usually be solved faster.

Total running time of edit distance

The total running time is $O(n^2)$.

Question No: 49 (Marks: 5)

What is the cost of the following graph?



Cost = 33

Question No: 50 (Marks: 5)

Let the adjacency list representation of an undirected graph is given below. Explain what general property of the list indicates that the graph has an isolated vertex.

a → b → c → e
b → a → d
c → a → d → e → f
d → b → c → f
e → a → c → f
f → c → d → e
g

If any of the vertex do not have any adjacent vertices, It is shown in the list a single entity.

Question No: 48 (Marks: 3)

Recursive explanation of dynamic programming.

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming

algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller sub problems.
- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

Question No: 28 (Marks: 2)

Differentiate between back edge and forward edge.

Back edge: (u, v) where v is an ancestor of u in the tree.

Forward edge: (u, v) where v is a proper descendent of u in the tree.

Question No: 29 (Marks: 2)

How the generic greedy algorithm operates in minimum spanning tree?

A generic greedy algorithm operates by repeatedly adding any *safe* edge to the current spanning tree.

Question No: 30 (Marks: 2)

What are two cases for computing assuming we already have the previous matrix $d^{(k-1)}$ using Floyd-Warshall algorithm?

There are two basic cases:

1. Don't go through vertex k at all.
2. Do go through vertex k .

Question No: 31 (Marks: 3)

Describe Minimum Spanning Trees Problem with examples.

A common problem is communications networks and circuit design is that of connecting together a set of nodes by a network of total minimum length. The length is the sum of lengths of connecting wires.

Question No: 32 (Marks: 3)

What is decision problem, also explain with example?

A problem is called a *decision problem* if its output is a simple “yes” or “no” (or you may think of this as true/false, 0/1, accept/reject.)

Question No: 33 (Marks: 3)

Prove that the generic TRAVERSE (S) marks every vertex in any connected graph exactly once and the set of edges $(v, \text{parent}(v))$ with $\text{parent}(v) \neq S$ form a spanning tree of the graph.

First, it should be obvious that no vertex is marked more than once. Clearly, the algorithm marks S . Let $v \neq S$ be a vertex and let $S = u_0, u_1, \dots, u_k = v$ be a path from S to v with the minimum number of edges.

Page #125 lemma

Question No: 35 (Marks: 5)

Prove the following lemma,

Lemma: Given a digraph $G = (V, E)$, consider any DFS forest of G and consider any edge $(u, v) \in E$. If this edge is a tree, forward or cross edge, then $f[u] > f[v]$. If this edge is a back edge, then $f[u] \leq f[v]$

Proof: For the non-tree forward and back edges the proof follows directly from the parenthesis lemma. For example, for a forward edge (u, v) , v is a descendent of u and so v 's start-finish interval is contained within u 's implying that v has an earlier finish time. For a cross edge (u, v) we know that the two time intervals are disjoint. When we were processing u , v was not white (otherwise (u, v) would be a tree edge), implying that v was started before u . Because the intervals are disjoint, v must have also finished before u .

1)RAM(Random Access memory)and its Applications?

A RAM is an idealized machine with an infinitely large random-access memory. Instructions are executed one-by-one (there is no parallelism). Each instruction involves performing some basic operation on two values in the machine's memory. Basic operations include things like assigning a value to a variable, computing any basic arithmetic operation (+, -, $\times$, integer division) on integer values of any size, performing any comparison (e.g. $x \leq 5$) or boolean operations, accessing an element of an array (e.g. $A[10]$). We assume that each basic operation takes the same constant time to execute.

2) Describe Dijkstra's algorithm working?

Dijkstra's algorithm works on a weighted directed graph $G = (V, E)$ in which all edge weights are non-negative, i.e., $w(u, v) \geq 0$ for each edge $(u, v) \in E$.

3) Prim algorithm graph?

Prim's algorithm builds the MST by adding leaves one at a time to the current tree. We start with a root vertex r ; it can be any vertex. At any time, the subset of edges A forms a single tree. We look to add a single vertex as a leaf to the tree. Graph on page #150.

4) Give a detailed example for 2-d maxima problem

Suppose you want to buy a car. You want to pick the fastest car. But fast cars are expensive; you want the cheapest. You cannot decide which is more important: speed or price. Definitely do not want a car if there is another that is both faster and cheaper. We say that the fast, cheap car *dominates* the slow, expensive car relative to your selection criteria. So, given a collection of cars, we want to list those cars that are not dominated by any other. Here is how we might model this as a formal problem.

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to be dominated by point q if $p.x \leq q.x$ and $p.y \leq q.y$.
- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be maximal if it is not dominated by any other point in P .

The car selection problem can be modelled this way: For each car we associate (x, y) pair where x is the speed of the car and y is the negation of the price. High y value means a cheap car and low y means expensive car. Think of y as the money left in your pocket after you have paid for the car. Maximal points correspond to the fastest and cheapest cars.

The *2-dimensional Maxima* is thus defined as

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space, output the set of maximal points of P , i.e., those points p_i such that p_i is not dominated by any other point of P .

8) Pseudo code algorithm for DFS Timestamp?

DFS(G)

```
1 for (each  $u \in V$ )
2 do color[ $u$ ] = white
3 pred[ $u$ ] = nil
4 time = 0
5 for each  $u \in V$ 
6 do if (color[ $u$ ] = white)
7 then DFSVISIT( $u$ )
```

(2) what is common problem in communication networks and circuit designing? 2

A common problem in communications networks and circuit design is that of connecting together a set of nodes by a network of total minimum length. The length is the sum of lengths of connecting wires.

(3) write pseudo code of relaxing a vertex 5

RELAX(u, v)

1 if ($d[u] + w(u, v) < d[v]$)

2 then $d[v] \leftarrow d[u] + w(u, v)$

3 $pred[v] = u$

Page #155

(4) write pseudo code of dijkstra algorithm? 5

Page #156

(5) define NP completeness 5

The set of NP-complete problems is all problems in the complexity class NP for which it is known that if any one is solvable in polynomial time, then they all are. Conversely, if any one is not solvable in polynomial time, then none are.

Definition: A decision problem L is NP-Hard if

$L_0 \leq_P L$ for all $L_0 \in NP$.

Definition: L is NP-complete if

1. $L \in NP$ and

2. $L_0 \leq_P L$ for some known NP-complete problem L_0 .

Page #176

(6) define Floyd Warshall algorithm in these two cases 5

do not go through a vertex k at all

do go through a vertex k

Don't go through k at all

Then the shortest path from i to j uses only intermediate vertices $\{1, 2, \dots, k-1\}$. Hence the length of the shortest is $d(k-1)$

ij

Do go through k

First observe that a shortest path does not go through the same vertex twice, so we can assume that we pass through k exactly once. That is, we go from i to k and then from k to j . In order for the overall path to be as short as possible, we should take the shortest path from i to k and the shortest path from k to j . Since each of these paths uses intermediate vertices $\{1, 2, \dots, k-1\}$, the length of the path is

$d(k-1)$

$ik + d(k-1)$

kj .

Page #162

(7) Define DAG 3

A directed graph that is acyclic is called a *directed acyclic graph (DAG)*.

4 Questions of 5 Marks

1) Write steps of sieve techniques

We then *partition* A into three parts.

1. $A[q]$ contains the pivot element x ,
2. $A[1..q - 1]$ will contain all the elements that are less than x and
3. $A[q + 1..n]$ will contains all elements that are greater than x .

Page #36

Prove the Lemma:

Consider a diagraph $G = (V, E)$ and any DFS forest for G . G has a cycle if and only if the DFS forest has a back edges

Proof: If there is a back edge (u, v) then v is an ancestor of u and by following tree edge from v to u , we get a cycle.

Where the cliquer cover problem is used?

In clustering applications.

2) Analyze the brute force maxima algorithm

Assume that the input size is n , and for the running time we will count the number of time that any element of P is accessed. Clearly we go through the outer loop n times, and for each time through this loop, we go through the inner loop n times as well. The condition in the if-statement makes four accesses to P . The output statement makes two accesses for each point that is output. In the worst case every point is maximal so these two access are made for each time through the outer loop.

3) How Kruskal's algorithm works ?

Kruskal's algorithm works by adding edges in increasing order of weight (lightest edge first). If the next edge does not induce a cycle among the current set of edges, then it is added to A . If it does, we skip it and consider the next in order. As the algorithm runs, the edges in A induce a forest on the vertices. The trees of this forest are eventually merged until a single tree forms containing all vertices.

4) Which points should be supposed to prove the correctness of the Dijkstra's Algorithm

We will prove the correctness of Dijkstra's algorithm by Induction. We will use the definition that $_ (s, v)$

denotes the minimal distance from s to v .

For the base case

1. $S = \{s\}$
2. $d(s) = 0$, which is $_ (s, s)$

Page #158

What are two steps generally involved while developing a dynamic programming algorithm?

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller subproblems.
- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

What is the running time of the Dijkstra's algorithm?

$O(E \log V)$

The following adjacency matrix represents a graph that consists of four vertices labeled 0, 1, 2 and 3. The entries in the matrix indicate edge weights.

	0	1	2	3
0	0	1	0	3
1	2	0	4	0
2	0	1	0	1
3	2	0	0	0

Answer the following question:

Can an adjacency matrix for a directed graph ever not be square in shape? Why or why not?

NO. Because number of rows and number of columns are always same.

What is the application of edit distance technique?

Edit Distance: Applications

There are numerous applications of the Edit Distance algorithm. Here are some examples:

Spelling Correction

If a text contains a word that is not in the dictionary, a 'close' word, i.e. one with a small edit distance, may be suggested as a correction. Most word processing applications, such as Microsoft Word, have spelling checking and correction facility. When Word, for example, finds an incorrectly spelled word, it makes suggestions of possible replacements.

Plagiarism Detection

If someone copies, say, a C program and makes a few changes here and there, for example, change variable names, add a comment or two, the edit distance between the source and copy may be small. The edit distance provides an indication of similarity that might be too close in some situations.

Computational Molecular Biology

Speech Recognition

Algorithms similar to those for the edit-distance problem are used in some speech recognition systems. Find a close match between a new utterance and one in a library of classified utterances.

7. What are the minimum and maximum numbers of elements in a heap of height h ?

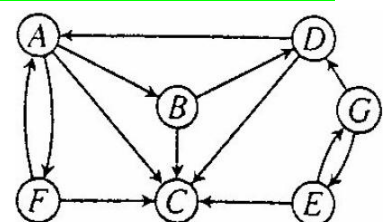
$\min = 2^h$

$\max = 2^{(h+1)} - 1$

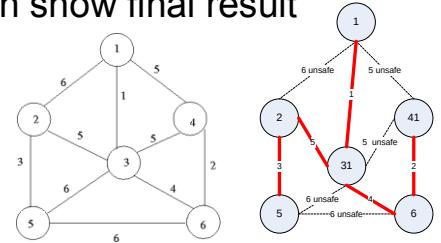
8. Where clique cover problem arises?

The clique cover problem arises in applications of clustering. We put an edge between two nodes if they are similar enough to be clustered in the same group. We want to know whether it is possible to cluster all the vertices into k groups.

9. Show the strongly connected components of the following graph using DFS algorithm. Take node E as a starting node. [You can show final result in exam software and need not to show all intermediate steps].



10. Apply Kruskal's algorithms on the following graph.[You can show final result in exam software and need not to show all intermediate steps].



8. Apply Kruskal's algorithms on the following graph.[You can show final result in exam software and need not to show all intermediate steps].

5) a table is given and we have to make the adjacency list and matrix

6) an mst graph is given and we have to calculate its weight

Q1) answer the following according to Floyd Marshall 1) running time 2) space use

6) Floyd-Warshall matrix diya tha os py 3 iteration apply karny thy 5marks

1- Given a graph run dijkstra's algorithm to find shortest path from vertex "S" to all other vertices (5 marks)

2- Given a graph run prim's algorithm to find minimum spanning tree. Only show the order of edges to be added to make spanning tree (5 marks)

3- Given a graph run DFS and label timestamping (5 marks)

8- Given an adjacency list, make the graph. State whether the graph is cyclic or not? (3 marks)

Is Dijkstra's algorithm a dynamic programming problem. Answer with yes or no with reason

Question No: 34 (Marks: 5)

Suppose you could prove that an NP-complete problem can not be solved in polynomial time. What would be the consequence?

Question No: 51 (Marks: 5)

Floyd Warshall algorithm with three recursive steps

5) Floyd-Warshall matrix complexity?

6) Convert Shortest path in to single source shortest path problem?

1) Given a matrix of graph G

Is G directed or undirected graph ?

0 1 2 3

Answer yes or no and give a brief explanation for your choice. If

0 0 1 0 3

problem A reduces (is polynomial-time reducible) to problem B and B

1 2 0 4 0

is NP-complete then A is NP-complete

2 0 1 0 1

5. Consider the following two problems. In P1 we are given as input a

3 2 0 0 0

set of n squares (specified by their corner points), and a number k.

The problem is to determine whether there is any point in the plane

that is covered by k or more squares. In P2 we are given as input an n-vertex graph,

and a number k; the problem is to determine whether there is a set of k mutually

adjacent vertices. (E.g. for k = 3 we are just looking for a triangle in the graph.).

Obviously, the problems are both in NP. There exists a simple translation from P1 to

P2: just make a graph vertex for each square, and add an edge between a pair of

vertices if the corresponding two squares overlap. If P1 is NP-complete, would this

translation imply that P2 is NP-complete? (Give your Answer in Yes or No)

